Lec-12

12/10/26 5:05 PM

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Example 1:

```
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
```

1 2 3 4 5 6 7

Example 2:

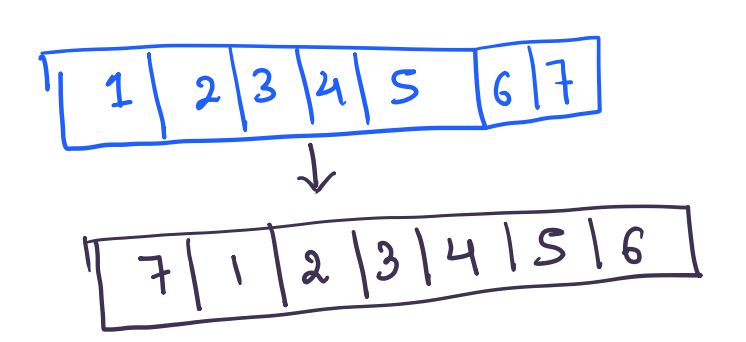
```
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]
```

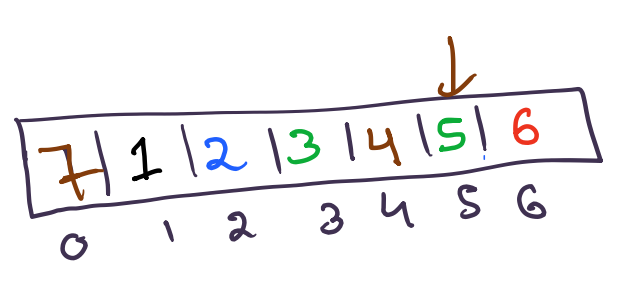
[1, 2, 3, 4, 5, 6, 7]

K = 1 7 1 2 3 4 5 6
K = 2 6 7 1 2 3 4 5
K = 3 5 6 7 1 2 3 4
K = 4 4 5 6 7 1 2 3
K = 5 3 4 5 6 7 1 2
K = 6 2 3 4 5 7 6 1
K = 7 1 2 3 4 5 6 7
K = 8 7 1 2 3 4 5 6
K = 9 6 7 1 2 3 4 5
K = 10 5 6 7 1 2 3 4
K = 11 4 5 7 6 7 1 2

① K = K % N ①

[1, 2, 3, 4, 5, 6, 7]





N = arr.len - 1

① item = arr[N-1] → 6

ii for(i = N-2 i >= 0 i--){
arr[i+1] = arr[i]

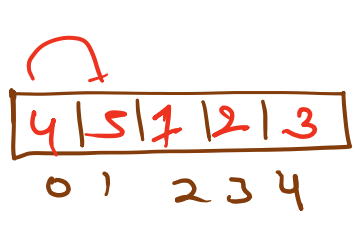
i = 5 arr[6] = arr[5]
i = 4 arr[5] = arr[4]
i = 3 arr[4] = arr[3]
i = 2 arr[3] = arr[2]
i = 1 arr[2] = arr[1]
i = 0 arr[1] = arr[0]

iii arr[0] = item

```java
public static void main(String[] args) {
    // TODO Auto-generated method stub
    int[] arr = { 1, 2, 3, 4, 5, 6, 7 };
    int k = 3;
    Rotate(arr, k);
    for (int i = 0; i < arr.length; i++) {
        System.out.print(arr[i] + " ");
    }
}

public static void Rotate(int[] arr, int k) {

    int n = arr.length;
    k = k % n;
    for (int j = 1; j <= k; j++) {
        int item = arr[n - 1];
        for (int i = n - 2; i >= 0; i--) {
            arr[i + 1] = arr[i];
        }
        arr[0] = item;
    }
}
```

5

K = 2 % 5 = 2



K = 2

item = arr[4] → 4

i = 3 arr[2] = arr[3]
i = 2 arr[3] = arr[2]
i = 1 arr[2] = arr[1]
i = 0 arr[1] = arr[0]

T L C

Reversed Algo

[1, 2, 3, 4, 5, 6, 7] → [1 2 3 4 7 6 5]

N-K K

(4 3 2 1 7 6 5)

K = 3 5 6 7 1 2 3 4

5 6 7 1 2 3 4

① Starting → N-K element → i = 0, j = N-K-1
ⅱ Last → K element → i = N-K j = N-1

[1, 2, 3, 4, 5, 6, 7] K = 3, N = 7

N-K = 7-3 = 4

```java
public static void Rotate(int[] arr, int k) {

    int n = arr.length;
    k = k % n;
    // 1st n-k element reverse krna hai
    Reverse(arr, 0, n - k - 1);
    // last  k element reverse krna hai
    Reverse(arr, n-k, n -  1);
    // saare element reverse krna hai
    Reverse(arr, 0, n-1);
}
```

0, 3

[1, 2, 3, 4, 5, 6, 7]
0 1 2 3 4 5 6

4 3 2 1 7 6 5

5 6 7 1 2 3 4

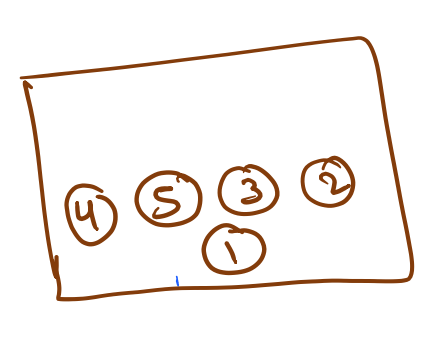
Sorting

[4 5 3 2 1] 1 2 3 4 5

① bubble
ⅱ selection
ⅲ Insertion
ⅳ D.N.F
ⅴ counting
ⅵ Merge sort
ⅵ Heap sort
ⅶ Quick sort
Randomize quick sort
cycle sort

[4, 5, 3, 2, 1]

① bubble sort [4 5 3 2 1]



① 4 5 3 2 1
4 5 3 2 1
4 3 5 2 1
4 3 2 5 1
4 3 2 1 5

ⅱ 4 3 2 1 5
3 4 2 1 5
3 2 4 1 5
3 2 1 4 5

ⅲ 3 2 1 4 5
2 3 1 4 5
2 1 3 4 5

ⅳ 2 1 3 4 5
1 2 3 4 5

for(Turn = 1 turn < N; turn++){
for(i = 0 i < 3, i++){
if(arr[i] > arr[i+1]){
swap(i, i+1)
}
}
}

arr.len - Turn ④

for(Turn = 1 turn < N; turn++){
for(i = 0 i < 3, i++){
if(arr[i] > arr[i+1]){
swap(i, i+1)
}
}
}

①

[4, 5, 3, 2, 1]
0 1 2 3 4

i = 0 [4 5 3 2 1]
0 1 2 3 4
i = 1 [4 3 5 2 1]
0 1 2 3 4
i = 2 [4, 3, 2, 5, 1]
0 1 2 3 4
i = 3 [4 3 2 1 5]
0 1 2 3 4

ⅱ [4, 3, 2, 1, 5]
i = 0 [3, 4, 2, 1, 5]
0 1 2 3 4
i = 1 [3, 2, 4, 1, 5]
0 1 2 3 4
i = 2 [3, 2, 1, 4, 5]
0 1 2 3 4

ⅲ [3, 2, 1, 4, 5]
0 1 2 3 4
i = 0 [2 3 1 4 5]
0 1 2 3 4
i = 1 [2, 1, 3, 4, 5]

ⅳ [2, 1, 3, 4, 5]
0 1 2 3 4
i = 0 [1, 2, 3, 4, 5]

0 1 2 3 4 5 6
[4, -1, 5, 3, 2, 1, 7]

```java
public static void main(String[] args) {
    // TODO Auto-generated method stub
    int[] arr = { 4, -1, 5, 3, 2, 1, 7 };
    System.out.println(min_form_ith_index(arr, 2));
}
public static int min_form_ith_index(int[] arr, int idx) {
}
```

[4, -1, 5, 3, 2, 1, 7]
0 1 2 3 4 5 6

Mini = idx → 2

for(i = idx+1 i < N i++){
if(arr[i] < arr[mini]){
mini = i
}
}

```java
int[] arr = { 4, -1, 5, 3, 2, 1, 7 };
```

i = 0 [4, -1, 5, 3, 2, 1, 7]
0 1 2 3 4 5 6 mini = 1

i = 1 [-1, 4, 5, 3, 2, 1, 7] → mini = 5
0 1 2 3 4 5 6
[-1 1 5 3 2 4 7]
0 1 2 3 4 5 6 mini = 4

i = 2 [-1, 1, 2 3 5 4 7]
0 1 2 3 4 5 6 min = 3

i = 3 [-1, 1, 2, 3, 5 4 7]
0 1 2 3 4 5 6 mini = 5

i = 4 [-1 1 2 3 4 5 7]
0 1 2 3 4 5 6 mini = 5

i = 5

[-1 1 2 3 4 5 7]
0 1 2 3 4 5 6
i = 6

4 3 2 1

$\frac{n \times (n-1)}{2}$

```java
int[] arr = { 1, 2, 7, 8, 9, 11, 4 };
```

→ Sort

1 2 7 8 9 4 11
1 2 7 8 4 9 11
0 1 2 3 4 5 6
1 2 7 4 8 9 11
1 2 4 7 8 9 11

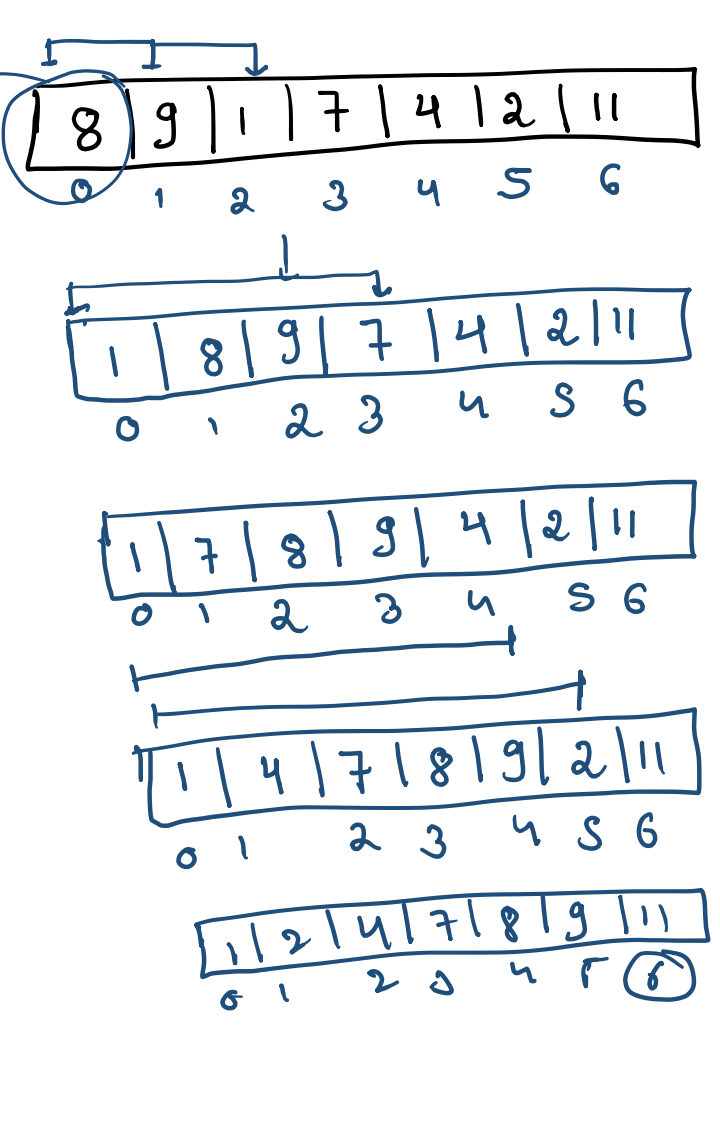
item = arr[i]
j = i-1
while(j >= 0 && arr[j] > item){
arr[j+1] = arr[j]
arr[j] = item
j--
}

j+1

[3, 4, 5, 6, 2]
0 1 2 3 4
i = 4
j = 3
item = 2
6 > 2
[3 4 5 2 6]
0 1 2 3 4
[3, 4, 2, 5 6]
[3 2 4 5 6]
[2 3 4 5 6]
J = -1

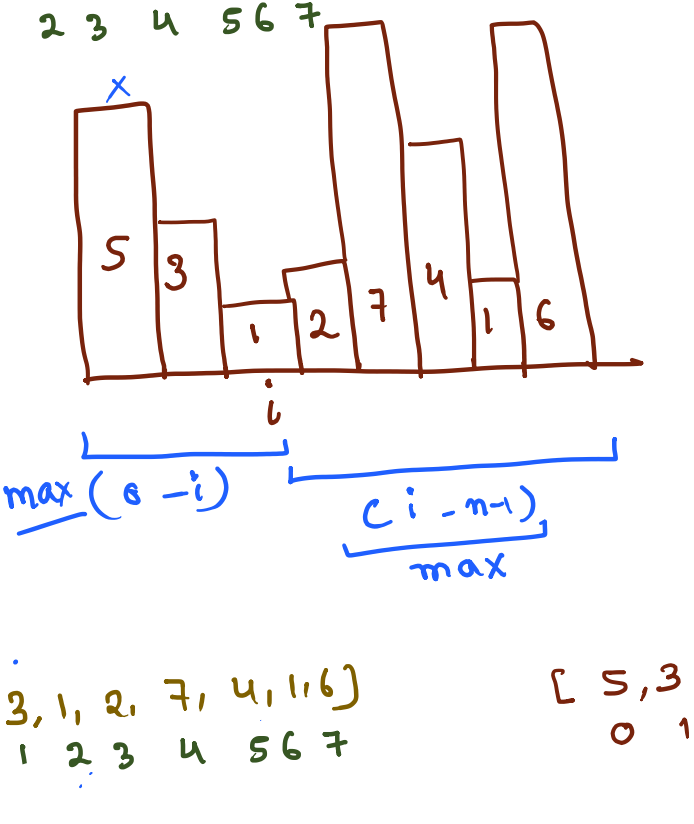
```java
public static void main(String[] args) {
    // TODO Auto-generated method stub
    int[] arr = { 8, 9, 1, 7, 4, 2, 11 };
    // InsertLastElement(arr, arr.length - 1);
    Sort(arr);
    for (int i = 0; i < arr.length; i++) {
        System.out.print(arr[i] + " ");
    }
}

public static void Sort(int[] arr) {


}

public static void InsertLastElement(int[] arr, int i) {
    // TODO Auto-generated method stub
    int item = arr[i];
    int j = i - 1;
    while (j >= 0 && arr[j] > item) {
        arr[j + 1] = arr[j];
        arr[j] = item;
        j--;
    }
    // correct index is j+1;
}
```

1 ②

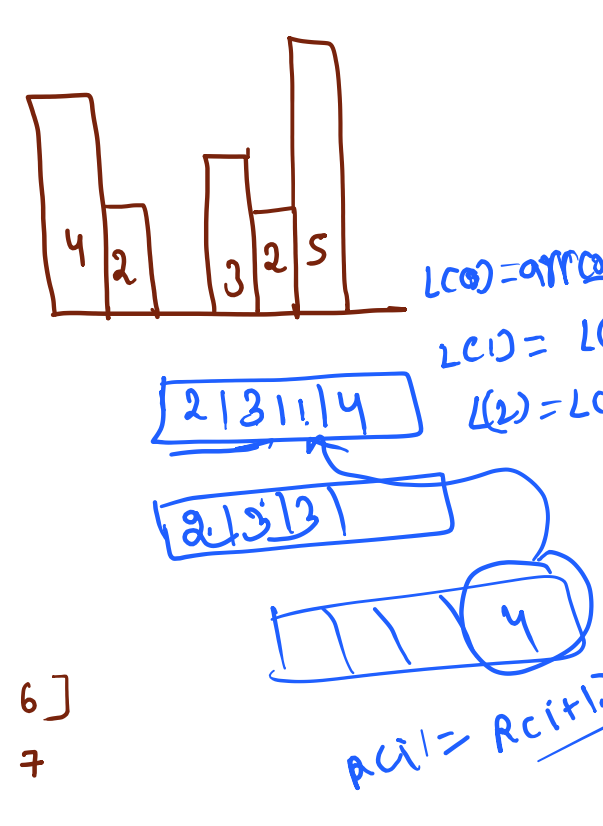


```
height = [4,2,0,3,2,5]
```

[5, 3, 1, 2, 7, 4, 1, 6]
0 1 2 3 4 5 6 7



max(0 - i) (i - n-1) max



L[0] = arr[0]
L[1] = L[0], arr[1]
L[2] = L[1], arr[2]

R[i] = R[i+1], arr[i]

Sum += min(L[i], R[i]) - arr[i]

[5, 3, 1, 2, 7, 4, 1, 6]
0 1 2 3 4 5 6 7

Left 5 5 5 5 7 7 7 7
0 1 2 3 4 5 6 7

Right 7 7 7 7 7 6 6 6
0 1 2 3 4 5 6 7

left[i] left[i-1] + arr[i]

[5, 3, 1, 2, 7, 4, 1, 6]
0 1 2 3 4 5 6 7

i = 0 min(left[0], R[0]) - arr[0] = min(5, 7) - 5 = 0
i = 1 min(L[1], R[1]) - arr[1] = min(5, 7) - 3 = 2
i = 2 min(L[2], R[2]) - arr[2] = min(5, 7) - 1 = 4
i = 3 min(L[3], R[3]) - arr[3] = min(5, 7) - 2 = 3
i = 4 min(L[4], R[4]) - arr[4] = min(7, 7) - 7 = 0
i = 5 min(L[5], R[5]) - arr[5] = min(7, 6) - 4 = 2
i = 6 min(L[6], R[6]) - arr[6] = min(7, 6) - 1 = 5
i = 7 min(L[7], R[7]) - arr[7] = min(7, 6) - 6 = 0

0 1 2 3 4 5
[2, 3, -6, 5, -1, 4]

[0-0] 2 → 2
[0-1] 2 3 → 5
[0-2] 2 3 -6 → -1
[0-3] 2 3 -6 5 → 4
[0-4] 2 3 -6 5 -1 → 3
[0-5] 2 3 -6 5 -1 4 → 7

[1-1] 3 → 3
[1-2] 3 -6 → -3
[1-3] 3 -6 5 → 2
[1-4] 3 -6 5 -1 → 1
[1-5] 3 -6 5 -1 4 → 5

2,2 -6 → -6
2,3 -6 5 → -1
2,4 -6 5 -1 → -2
-6 5 -1 4
2,5 → 2

3,3 5 → 5
3,4 5 -1 → 4
5 -1 4
3,5 → 8

4,4 -1 -1
-1 4
4,5 → 3

5,5 4 → 4

ans = -∞

for(i = 0 i < n i++){
int sum = 0
for(j = i j < n j++){
sum += arr[j]
ans = max(ans, sum)
}
}